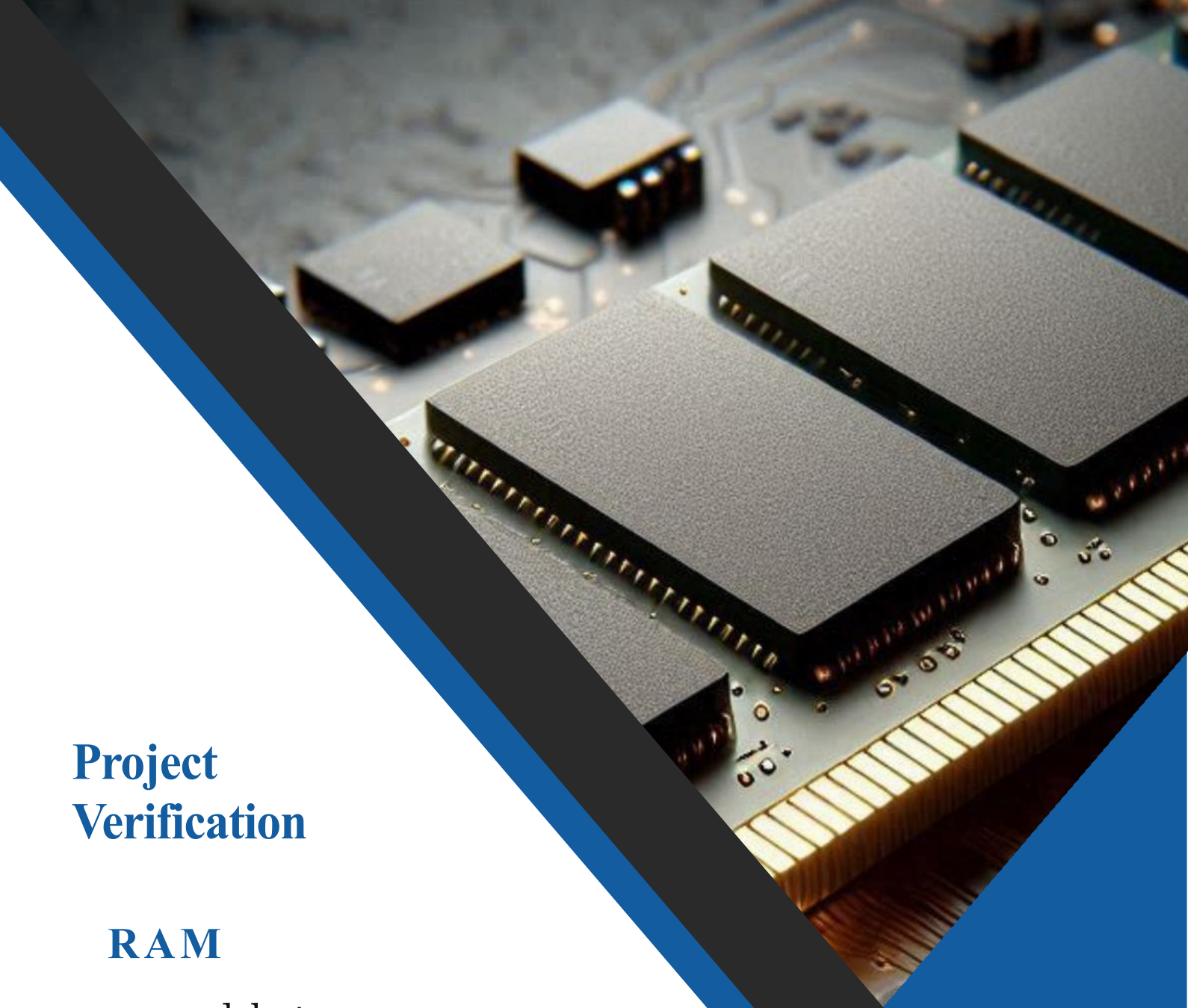


Project Verification

RAM

prepared by:

Mohamed Lotfy Mohamed

Under supervision of:

Eng. Kareem Wassem

Contents

1. Design RTL

1.1. Bugs detected

1.1.1. Read data functionality

- When (din[9:8] = 2'b11) in the read data mode, Ram took the data from the write address instead of the read address (It should be dout <= MEM[Rd_Addr] Instead of dout <= MEM[Wr_Addr]).

1.1.2. tx_valid functionality

- tx_valid should be high during the read data operation not for only 1cycle, and after the read operatio, tx_valid should be low.

1.2. Design assertions

```
module ram_SVA(clk,rst_n,din,rx_valid,tx_valid,dout);

// interface input
input clk;
// Design input ports
input  [9:0] din;
input  rst_n, rx_valid;
// Design output ports
input reg [7:0] dout;
input reg tx_valid;
// Reset
property P1;
    (@(posedge clk) !rst_n |=> ((dout == 8'b0) && (tx_valid == 1'b0)));
endproperty
reset: assert property(P1);
cover property(P1);

// tx_valid deasserted during address/data phases
property P2;
    disable iff (!rst_n)
    (@(posedge clk)
    (rx_valid && ((din[9:8] == 2'b00) || (din[9:8] == 2'b01) || (din[9:8] == 2'b10)))
    |=> (tx_valid == 1'b0));
endproperty
tx_valid_low: assert property(P2);
cover property(P2);

// tx_valid rises after read_data and falls after 1 cycle
property P3;
    disable iff (!rst_n)
    (@(posedge clk)
    (rx_valid && (din[9:8] == 2'b11))
    |=> (tx_valid == 1'b1) ##1 $fell(tx_valid)[->1]);
endproperty
tx_valid_rise_fall: assert property(P3);
cover property(P3);

// Write Address eventually followed by Write Data
property P4;
    disable iff (!rst_n)
    (@(posedge clk)
    (rx_valid && (din[9:8] == 2'b00))
    |=>(rx_valid && (din[9:8] == 2'b01))[->1]);
endproperty
wa_wd: assert property(P4);
cover property(P4);

// Read Address eventually followed by Read Data
property P5;
    disable iff (!rst_n)
```

```
   (@(posedge clk)
    (rx_valid && (din[9:8] == 2'b10))
    |=>(rx_valid && (din[9:8] == 2'b11))[->1]);
endproperty
ra_rd: assert property (P5);
cover property (P5);

endmodule
```

1.3. RTL code

```
import shared_pkg::*;
module RAM (clk,rst_n,din,rx_valid,tx_valid,dout);

// interface input
input clk;
// Design input ports
input  [9:0] din;
input  rst_n, rx_valid;
// Design output ports
output reg [7:0] dout;
output reg tx_valid;

reg [ADDR_SIZE-1:0] Rd_Addr, Wr_Addr;
reg [ADDR_SIZE-1:0] MEM [MEM_DEPTH-1:0];

always @(posedge clk) begin
    if (~rst_n) begin
        dout <= 0;
        tx_valid <= 0;
        Rd_Addr <= 0;
        Wr_Addr <= 0;
    end
    else begin
        if (rx_valid) begin
            case (din[9:8])
                2'b00 :begin
                        Wr_Addr <= din[7:0];
                        tx_valid <= 0;
                end
                2'b01 :begin
                        MEM[Wr_Addr] <= din[7:0];
                        tx_valid <=0;
                end
                2'b10 :begin
                        Rd_Addr <= din[7:0];
                        tx_valid <=0;
                end
                2'b11 :begin
                        dout <= MEM[Rd_Addr];      // Rd_Addr not Wr_Addr; BUG 1
                        tx_valid <=1;
                end
                default : dout <= 0;
            endcase
        end
        //tx_valid <= (din[9] && din[8] && rx_valid)? 1'b1 : 1'b0; BUG 2
    end
end
endmodule
```

1.4. Golden model code

```
import shared_pkg::*;
module Single_port_SYNC_RAM (clk,rst_n,din,rx_valid,tx_valid_golden,dout_golden);

reg [ADDR_SIZE-1:0] MEM [MEM_DEPTH-1:0];
reg [ADDR_SIZE-1:0] Rd_Addr, Wr_Addr;
// interface input
input clk;
// Design input ports
input  [9:0] din;
input  rst_n, rx_valid;
// Design output ports
output reg [7:0] dout_golden;
output reg tx_valid_golden;
 // GENERATE MEMORY
 reg [ADDR_SIZE-1:0] RAM [MEM_DEPTH-1:0];
 // DEFINE 2 ADDRESSES ONE FOR WRITE AND ONE FOR READ
reg [ADDR_SIZE-1:0] wr_address , rd_address ;
 // DESIGN IMPLEMENTATION
 always @(posedge clk) begin
    if(~rst_n)begin
      dout_golden <= 0;
      tx_valid_golden <= 0;
      wr_address <= 0;
      rd_address <= 0;
    end
    else begin
        if(rx_valid)begin
          case (din[9:8])
            2'b00:begin
                  wr_address <= din[7:0];
                  tx_valid_golden <= 0;
            end
            2'b01:begin
                  RAM[wr_address] <= din[7:0];
                  tx_valid_golden <= 0;
            end
            2'b10:begin
                  rd_address <= din[7:0];
                  tx_valid_golden <= 0;
            end
            2'b11:begin
                   dout_golden <= RAM[rd_address];
                   tx_valid_golden <= 1;
            end
            default: dout_golden <=0;
         endcase
        end
        else
            tx_valid_golden <=0;
    end
```

```
end
endmodule
```

2. Do file

run.do

```
vlib work
vlog -f src_files.list  +cover -covercells
vsim -voptargs=+acc work.TOP -cover -classdebug -uvmcontrol=all
coverage exclude -src ram.sv -line 42 -code s
coverage exclude -src ram.sv -line 42 -code b
do wave.do
coverage save RAM_Coverage.ucdb -onexit
run -all

#quit -sim
# Code Coverage Report
#vcover report RAM_Coverage.ucdb \-codeAll -details -annotate -output RAM_code_coverage_report.txt
#
##Assertion Coverage Report
#vcover report RAM_Coverage.ucdb \-assert -details -annotate -output RAM_assertion_coverage_report.txt
#
##Functional Coverage Report
#vcover report RAM_Coverage.ucdb \-cvg -details -annotate -output RAM_functional_coverage_report.txt
```

3. Verification files

3.1. Shared package

```
package shared_pkg;
    bit test_finished ;
    int error_count ;
    int correct_count ;
    parameter FIFO_WIDTH = 16;
    parameter FIFO_DEPTH = 8;
endpackage
```

3.2. Interface

```
interface ram_if(clk);
// interface input
input bit clk;
// Design input ports
bit [9:0] din;
bit rst_n, rx_valid;
// Design output ports
logic [7:0] dout;
logic tx_valid;

// Golden model output ports
logic [7:0] dout_golden;
logic tx_valid_golden;

modport DUT (input clk,rst_n,rx_valid,din,output tx_valid,dout);

modport GOLDEN (input clk,rst_n,rx_valid,din,output tx_valid_golden,dout_golden);

endinterface
```

3.3. Top module

```
import uvm_pkg::*;
`include "uvm_macros.svh"
import ram_test_pkg::*;
module TOP();
// clk generation
bit clk;
initial begin
    clk=0;
    forever begin
        #1 clk = ~ clk;
    end
end
// instantations
ram_if r_if(clk);

RAM DUT (r_if.clk,r_if.rst_n,r_if.din,r_if.rx_valid,r_if.tx_valid,r_if.dout);

Single_port_SYNC_RAM GOLDEN
(r_if.clk,r_if.rst_n,r_if.din,r_if.rx_valid,r_if.tx_valid_golden,r_if.dout_golden);
// Binding
bind RAM ram_SVA
ram_assertions(r_if.clk,r_if.rst_n,r_if.din,r_if.rx_valid,r_if.tx_valid,r_if.dout);
// run test enviroment
initial begin
    uvm_config_db#(virtual ram_if)::set(null,"uvm_test_top","ram_if",r_if);
```

```
    run_test("ram_test");
end
endmodule
```

3.4. RAM sequence item package

```
package ram_sequence_item_pkg;
import uvm_pkg::*;
`include "uvm_macros.svh"
class ram_sequence_item extends uvm_sequence_item;
    // register the class ram_sequence_item to the factory
    `uvm_object_utils(ram_sequence_item)
    // randomize the input data
    rand bit rst_n,rx_valid;
    rand bit [9:0] din;

    logic  tx_valid;
    logic  [7:0] dout;

    logic [7:0] dout_golden;
    logic tx_valid_golden;


    // constructor
    function new(string name = "ram_sequence_item");
        super.new(name);
    endfunction
    // use built in convert2string function
    function string convert2string();
        return
$sformatf("%s,rst_n=%b,rx_valid=%b,din=%d,tx_valid=%b,dout=%d,dout_golden=%d,tx_valid_gold
en=%b",super.convert2string(),
                        rst_n,rx_valid,din,tx_valid,dout,dout_golden,tx_valid_golden);
    endfunction
    // create convert2string_stimlus function
    function string convert2string_stimlus();
        return
$sformatf("%s,rst_n=%b,rx_valid=%b,din=%d",super.convert2string(),rst_n,rx_valid,din);
    endfunction
    // constraint
    bit [1:0] prev_din;

    constraint write_only_c{
        // reset constraint
        rst_n dist{0:/5,1:/95};
        // rx_valid constraint
        rx_valid dist{0:/5,1:/95};
        // write constraint
        if (prev_din == 2'b00){
            din[9:8] inside {2'b00, 2'b01};
        }
        else if(prev_din == 2'b01){
             din[9:8] == 2'b00;
```

```
        }
    }
    constraint read_only_c{
        // reset constraint
        rst_n dist{0:/5,1:/95};
        // rx_valid constraint
        rx_valid dist{0:/5,1:/95};
        // read constraint
        if (prev_din == 2'b10){
            din[9:8]  == 2'b11;
        }
        else if(prev_din == 2'b11){
            din[9:8] == 2'b10;
        }
    }
    constraint read_write_c{
        // reset constraint
        rst_n dist{0:/5,1:/95};
        // rx_valid constraint
        rx_valid dist{0:/5,1:/95};
        // write and read constraint
        if (prev_din == 2'b00){
            din[9:8] inside {2'b00, 2'b01};
        }
        else if (prev_din == 2'b01){
            din[9:8] dist {2'b10:/60, 2'b00:/40};
        }
        else if (prev_din == 2'b10){
            din[9:8] inside {2'b10, 2'b11};
        }
        else if (prev_din == 2'b11){
            din[9:8] dist {2'b00:/60, 2'b10:/40};
        }
    }
    function void post_randomize();
    prev_din = din[9:8]; // Save the current din[9:8]  for next transaction
endfunction

endclass
endpackage
```

3.5. RAM sequence package

```
package ram_sequence_pkg;
import ram_sequence_item_pkg::*;
import uvm_pkg::*;
`include "uvm_macros.svh"
// reset_sequence
class ram_reset_sequence extends uvm_sequence #(ram_sequence_item);
// register ram_reset_sequence class to the factory
`uvm_object_utils(ram_reset_sequence)
// create a handel form ram_sequence_item class
ram_sequence_item seq_item;
// constructor
function new(string name ="ram_reset_sequence");
    super.new(name);
endfunction
// body task
task body();
        // create object from ram_sequence_item class
        seq_item = ram_sequence_item::type_id::create("seq_item");
        start_item(seq_item);
        seq_item.rst_n = 0;
        seq_item.din =0;
package ram_sequence_pkg;
import ram_sequence_item_pkg::*;
import uvm_pkg::*;
`include "uvm_macros.svh"
// reset_sequence
// Ram_1
class ram_reset_sequence extends uvm_sequence #(ram_sequence_item);
// register ram_reset_sequence class to the factory
`uvm_object_utils(ram_reset_sequence)
// create a handel form ram_sequence_item class
ram_sequence_item seq_item;
// constructor
function new(string name ="ram_reset_sequence");
    super.new(name);
endfunction
// body task
task body();
        // create object from ram_sequence_item class
        seq_item = ram_sequence_item::type_id::create("seq_item");
        start_item(seq_item);
        seq_item.rst_n = 0;
        seq_item.din =0;
        seq_item.rx_valid =0;
        finish_item(seq_item);
endtask
endclass
```

```
// write_only_sequence
// Ram_2
class ram_write_only_sequence extends uvm_sequence #(ram_sequence_item);
// register ram_write_only_sequence class to the factory
`uvm_object_utils(ram_write_only_sequence)
// create a handel form ram_sequence_item class
ram_sequence_item seq_item;
// constructor
function new(string name ="ram_write_only_sequence");
    super.new(name);
endfunction
// body task
task body();
    // create object from ram_sequence_item class
    seq_item = ram_sequence_item::type_id::create("seq_item");
    repeat(10000)begin
        start_item(seq_item);
        seq_item.constraint_mode(0);                // disable all constraints
        seq_item.write_only_c.constraint_mode(1);   // enable write constraint
        assert(seq_item.randomize());
        finish_item(seq_item);
    end
endtask
endclass

// read_only_sequence
// Ram_3
class ram_read_only_sequence extends uvm_sequence #(ram_sequence_item);
// register ram_read_only_sequence class to the factory
`uvm_object_utils(ram_read_only_sequence)
// create a handel form ram_sequence_item class
ram_sequence_item seq_item;
// constructor
function new(string name ="ram_read_only_sequence");
    super.new(name);
endfunction
// body task
task body();
    // create object from ram_sequence_item class
    seq_item = ram_sequence_item::type_id::create("seq_item");
    repeat(10000)begin
        start_item(seq_item);
        seq_item.constraint_mode(0);                // disable all constraints
        seq_item.read_only_c.constraint_mode(1);   // enable read constraint
        assert(seq_item.randomize());
        finish_item(seq_item);
    end
endtask
endclass

// write_read_sequence
// Ram_4
```

```
class ram_write_read_sequence extends uvm_sequence #(ram_sequence_item);
// register ram_write_read_sequence class to the factory
`uvm_object_utils(ram_write_read_sequence)
// create a handel form ram_sequence_item class
ram_sequence_item seq_item;
// constructor
function new(string name ="ram_write_read_sequence");
    super.new(name);
endfunction
// body task
task body();
    // create object from ram_sequence_item class
    seq_item = ram_sequence_item::type_id::create("seq_item");
    repeat(50000)begin
        start_item(seq_item);
        seq_item.constraint_mode(0);                // disable all constraints
        seq_item.read_write_c.constraint_mode(1);   // enable read and write  constraint
        assert(seq_item.randomize());
        finish_item(seq_item);
    end
endtask
endclass
endpackage
```

3.6. RAM sequencer package

```
package ram_sequencer_pkg;
import ram_sequence_item_pkg::*;
import uvm_pkg::*;
`include "uvm_macros.svh"
class ram_sequencer extends uvm_sequencer #(ram_sequence_item);
// register ram_sequencer class to the factory
`uvm_component_utils(ram_sequencer);
// constructor
function new(string name = "ram_sequencer", uvm_component parent = null);
    super.new(name,parent);
endfunction
endclass

endpackage
```

3.7. RAM configuration package

```
package ram_config_pkg;
import uvm_pkg::*;
`include "uvm_macros.svh"
class ram_config extends uvm_object;
    `uvm_object_utils(ram_config)
    virtual ram_if ram_vif;
    uvm_active_passive_enum is_active;
    function new(string name = "ram_config");
        super.new(name);
    endfunction
```

```
endclass
endpackage
```

3.8. RAM coverage package

```
package ram_coverage_pkg;
import uvm_pkg::*;
import shared_pkg::*;
import ram_sequence_item_pkg::*;
`include "uvm_macros.svh"
class ram_coverage extends uvm_component;

`uvm_component_utils(ram_coverage)
uvm_analysis_export #(ram_sequence_item) cov_export;
uvm_tlm_analysis_fifo #(ram_sequence_item) cov_fifo;
ram_sequence_item seq_item_cov;

// group coverage
covergroup cvr_gp;
    // Check din[9:8] takes 4 possible values
    din_98:coverpoint seq_item_cov.din[9:8];
    // wCheck write data after write address
    wd_wa:coverpoint seq_item_cov.din[9:8] {
        bins wd_after_wa = (2'b00 => 2'b01);
    }
    // Check read data after read address
    rd_ra:coverpoint seq_item_cov.din[9:8] {
        bins rd_after_ra = (2'b10 => 2'b11);
    }
    // Check write address => write data => read address => read data
    din_trans:coverpoint seq_item_cov.din[9:8] {
        bins din_98_trans = (2'b00 => 2'b01 => 2'b10 => 2'b11);
    }
    // rx_valid
    rx_valid_cp:coverpoint seq_item_cov.rx_valid{
        bins rx_valid_value[] = {0,1};
    }
    // tx_valid
    tx_valid_cp:coverpoint seq_item_cov.tx_valid{
        bins tx_valid_value[] = {0,1};
    }
    // cross coverage
    cross din_98,rx_valid_cp{
        option.cross_auto_bin_max = 0;
        bins din_98_rx_valid = binsof(din_98) && binsof(rx_valid_cp) intersect {1};
    }

    cross din_98,tx_valid_cp{
        option.cross_auto_bin_max = 0;
        bins din_98_rx_valid = binsof(din_98) intersect {2'b11} && binsof(tx_valid_cp)
intersect {1};
    }
```

```
endgroup

function new(string name = "ram_coverage", uvm_component parent = null);
    super.new(name,parent);
    cvr_gp = new();
endfunction
// Build phase
function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    cov_export = new("cov_export",this);
    cov_fifo = new("cov_fifo",this);
endfunction
// connect phase
function void connect_phase(uvm_phase phase);
    super.connect_phase(phase);
    cov_export.connect(cov_fifo.analysis_export);
endfunction
// run phase
task run_phase(uvm_phase phase);
    super.run_phase(phase);
    forever begin
        cov_fifo.get(seq_item_cov);
        cvr_gp.sample();
    end

endtask

endclass
endpackage
```

3.9. RAM driver package

```
package ram_driver_pkg;
import ram_config_pkg::*;
import uvm_pkg::*;
import ram_sequence_item_pkg::*;
`include "uvm_macros.svh"

class ram_driver extends uvm_driver #(ram_sequence_item);
 // register ram_driver class to the factory
`uvm_component_utils(ram_driver)
// create handels
virtual ram_if ram_vif;
ram_sequence_item seq_item;
// constructor
function new(string name = "ram_driver", uvm_component parent = null);
    super.new(name,parent);
endfunction

//run phase
task run_phase(uvm_phase phase);
super.run_phase(phase);
forever begin
    seq_item =ram_sequence_item::type_id::create("seq_item");
    seq_item_port.get_next_item(seq_item);

    ram_vif.rst_n       = seq_item.rst_n;
    ram_vif.rx_valid    = seq_item.rx_valid;
    ram_vif.din         = seq_item.din;

    @(negedge ram_vif.clk);
    seq_item_port.item_done();
    `uvm_info("run_phase",seq_item.convert2string_stimlus(),UVM_HIGH)
end
endtask
endclass
endpackage
```

3.10. RAM environment package

```
package ram_env_pkg;
import ram_driver_pkg::*;
import ram_scoreboard_pkg::*;
import ram_agent_pkg::*;
import ram_coverage_pkg::*;
import uvm_pkg::*;
`include "uvm_macros.svh"
class ram_env extends uvm_env;
 // register ram_env class to the factory
`uvm_component_utils(ram_env)
// create handels
ram_agent agt;
ram_scoreboard sb;
ram_coverage cov;
// constructor
function new(string name = "ram_env",uvm_component parent = null);
  super.new(name,parent);
endfunction
// Build the driver in the build phase
function void build_phase(uvm_phase phase);
  super.build_phase(phase);
 agt = ram_agent::type_id::create("agt",this);
 sb = ram_scoreboard::type_id::create("sb",this);
 cov = ram_coverage::type_id::create("cov",this);
endfunction
function void connect_phase(uvm_phase phase);
  super.build_phase(phase);
  agt.agt_ap.connect(sb.sb_export);
  agt.agt_ap.connect(cov.cov_export);
endfunction
endclass
endpackage
```

3.11. RAM monitor package

```
package ram_monitor_pkg;
import ram_sequence_item_pkg::*;
import shared_pkg::*;
import uvm_pkg::*;
`include "uvm_macros.svh"
class ram_monitor extends uvm_monitor;
 // register ram_monitor class to the factory
`uvm_component_utils(ram_monitor)
// create handels
virtual ram_if ram_vif;
ram_sequence_item rsp_seq_item;
uvm_analysis_port #(ram_sequence_item) mon_ap;
// constructor
function new(string name = "ram_monitor", uvm_component parent = null);
    super.new(name,parent);
endfunction
// Build phase
function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    mon_ap = new("mon_ap",this);
endfunction
// run phase
task run_phase(uvm_phase phase);
    super.run_phase(phase);
    forever begin
        rsp_seq_item = ram_sequence_item::type_id::create("rsp_seq_item");
        @(negedge ram_vif.clk);

        rsp_seq_item.rst_n          = ram_vif.rst_n;
        rsp_seq_item.rx_valid       = ram_vif.rx_valid;
        rsp_seq_item.din            = ram_vif.din;
        rsp_seq_item.tx_valid       = ram_vif.tx_valid;
        rsp_seq_item.dout           = ram_vif.dout;

        rsp_seq_item.dout_golden    = ram_vif.dout_golden;
        rsp_seq_item.tx_valid_golden= ram_vif.tx_valid_golden;

        mon_ap.write(rsp_seq_item);
        `uvm_info("run_phase",rsp_seq_item.convert2string(),UVM_HIGH)
    end
endtask
endclass
endpackage
```

3.12. RAM agent package

```
package ram_agent_pkg;
import uvm_pkg::*;
import ram_config_pkg::*;
import ram_sequencer_pkg::*;
import ram_driver_pkg::*;
import ram_monitor_pkg::*;
import ram_sequence_item_pkg::*;

`include "uvm_macros.svh"
class ram_agent extends uvm_agent;
 // register ram_agent class to the factory
`uvm_component_utils(ram_agent)
// create handels
ram_sequencer sqr;
ram_driver drv;
ram_monitor mon;
ram_config ram_cfg;
uvm_analysis_port #(ram_sequence_item) agt_ap;

// constructor
function new(string name = "ram_agent",uvm_component parent = null);
  super.new(name,parent);
endfunction
// build phase
function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    if(!uvm_config_db #(ram_config)::get(this,"","CFG",ram_cfg))begin
        `uvm_fatal("build_phase","Error in build phase during configuration");
    end

   // if(ram_cfg.is_active == UVM_ACTIVE)begin
        sqr = ram_sequencer::type_id::create("sqr",this);
        drv = ram_driver::type_id::create("drv",this);
    //end
    mon = ram_monitor::type_id::create("mon",this);
    agt_ap = new("agt_ap",this);

endfunction
// connect phase
function void connect_phase(uvm_phase phase);
    super.connect_phase(phase);
     //if(ram_cfg.is_active == UVM_ACTIVE)begin
        drv.seq_item_port.connect(sqr.seq_item_export);
        drv.ram_vif = ram_cfg.ram_vif;
    //end
    mon.ram_vif = ram_cfg.ram_vif;
    mon.mon_ap.connect(agt_ap);
endfunction
endclass
endpackage
```

3.13. RAM test package

```
package ram_test_pkg;
import ram_env_pkg::*;
import ram_config_pkg::*;
import ram_agent_pkg::*;
import ram_sequencer_pkg::*;
import ram_sequence_pkg::*;
import uvm_pkg::*;
 `include "uvm_macros.svh"
 class ram_test extends uvm_test;
 // register ram_test class to the factory
`uvm_component_utils(ram_test);
// create handels
ram_config ram_cfg;
ram_reset_sequence reset_sequence;
ram_write_only_sequence write_only_sequence;
ram_read_only_sequence read_only_sequence;
ram_write_read_sequence write_read_sequence;
ram_env env;
virtual ram_if ram_vif;
// constructor
function new(string name = "ram_test", uvm_component parent = null);
    super.new(name,parent);
endfunction
// build phase
function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    env = ram_env::type_id::create("env",this);
    reset_sequence = ram_reset_sequence::type_id::create("reset_sequence",this);
    write_only_sequence =
ram_write_only_sequence::type_id::create("write_only_sequence",this);
    read_only_sequence =
ram_read_only_sequence::type_id::create("read_only_sequence",this);
    write_read_sequence =
ram_write_read_sequence::type_id::create("write_read_sequence",this);
    ram_cfg = ram_config::type_id::create("ram_cfg",this);

    if(!uvm_config_db#(virtual ram_if)::get(this,"","ram_if",ram_cfg.ram_vif))begin
      `uvm_fatal("build_phase","Error in build phase");
    end
    uvm_config_db#(ram_config)::set(this,"*","CFG",ram_cfg);
endfunction
// run phase
task run_phase(uvm_phase phase);
    super.run_phase(phase);
    phase.raise_objection(this);

    `uvm_info("run_phase","reset generation start",UVM_MEDIUM)
      reset_sequence.start(env.agt.sqr);
    `uvm_info("run_phase","reset deasserted",UVM_MEDIUM)
```

```
    `uvm_info("run_phase","WRITE generation start",UVM_MEDIUM)
      write_only_sequence.start(env.agt.sqr);
    `uvm_info("run_phase","WRITE generation END",UVM_MEDIUM)

    `uvm_info("run_phase","READ generation start",UVM_MEDIUM)
      read_only_sequence.start(env.agt.sqr);
    `uvm_info("run_phase","READ generation end",UVM_MEDIUM)

    `uvm_info("run_phase","READ AND WRITE generation start",UVM_MEDIUM)
      write_read_sequence.start(env.agt.sqr);
    `uvm_info("run_phase","READ AND WRITE generation end",UVM_MEDIUM)

    phase.drop_objection(this);
endtask
 endclass
endpackage
```

3.14. RAM scoreboard package

```
package ram_scoreboard_pkg;
import uvm_pkg::*;
import shared_pkg::*;
import ram_sequence_item_pkg::*;
`include "uvm_macros.svh"
int correct_count,error_count;
class ram_scoreboard extends uvm_scoreboard;
 // register ram_monitor class in the factory
`uvm_component_utils(ram_scoreboard)

uvm_analysis_export #(ram_sequence_item) sb_export;
uvm_tlm_analysis_fifo #(ram_sequence_item) sb_fifo;
ram_sequence_item seq_item_sb;


function new(string name = "ram_scoreboard", uvm_component parent = null);
    super.new(name,parent);
endfunction

function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    sb_export = new("sb_export",this);
    sb_fifo = new("sb_fifo",this);
endfunction

function void connect_phase(uvm_phase phase);
    super.build_phase(phase);
    sb_export.connect(sb_fifo.analysis_export);
endfunction

task run_phase(uvm_phase phase);
    super.run_phase(phase);
    forever begin
        sb_fifo.get(seq_item_sb);

        if((seq_item_sb.dout !== seq_item_sb.dout_golden) || (seq_item_sb.tx_valid !==
seq_item_sb.tx_valid_golden))begin
            `uvm_error("run_phase",$sformatf("comparison failed,transaction recived by the
dut:%s
                        while the dout_golden:%d and tx_valid_golden:%b",
                        seq_item_sb.convert2string(),seq_item_sb.dout_golden,seq_item_sb.t
x_valid_golden));
            error_count = error_count + 1;
        end
        else begin
            `uvm_info("run_phase",$sformatf("correct output
:%s",seq_item_sb.convert2string()),UVM_HIGH);
            correct_count = correct_count +1;
        end
    end
```

```
endtask
 function void report_phase(uvm_phase phase);
            super.report_phase(phase);
            `uvm_info("report_phase", $sformatf("correct_count = %d", correct_count),
UVM_LOW);
            `uvm_info("report_phase", $sformatf("error_count = %d", error_count),
UVM_LOW);
endfunction
endclass
endpackage
```

4. Table of Assertions

Feature	Assertion
ensures that whenever reset is asserted, the output signals (tx_valid and dout) are low	`(@(posedge clk) !rst_n |=> ((dout == 8'b0) && (tx_valid == 1'b0)));`

An assertion checks when the operation is (write_add_seq or write_data_seq or read_add_seq), the output tx_valid is low	`disable iff (!rst_n)`
	`(@(posedge clk)`
	`(rx_valid && ((din[9:8] == 2'b00) || (din[9:8] == 2'b01) || (din[9:8] == 2'b10)))`
	`|=> (tx_valid == 1'b0));`

An assertion checks that after a read_data_seq occurs, the tx_valid signal must high to indicate valid output and after 1 clk cycle it should eventually fall.	`disable iff (!rst_n)`
	`(@(posedge clk)`
	`(rx_valid && (din[9:8] == 2'b11))`
	`|=> (tx_valid == 1'b1) ##1 $fell(tx_valid)[->1]);`

An assertion checks that every Write Address operation must be eventually followed by a Write Data operation	`disable iff (!rst_n)`
	`(@(posedge clk)`
	`(rx_valid && (din[9:8] == 2'b00))`
	`|=>(rx_valid && (din[9:8] == 2'b01))[->1]);`

An assertion checks that every Read Address operation must be eventually followed by a Read Data operation	`disable iff (!rst_n)`
	`(@(posedge clk)`
	`(rx_valid && (din[9:8] == 2'b10))`
	`|=>(rx_valid && (din[9:8] == 2'b11))[->1]);`

5. Verification Plan

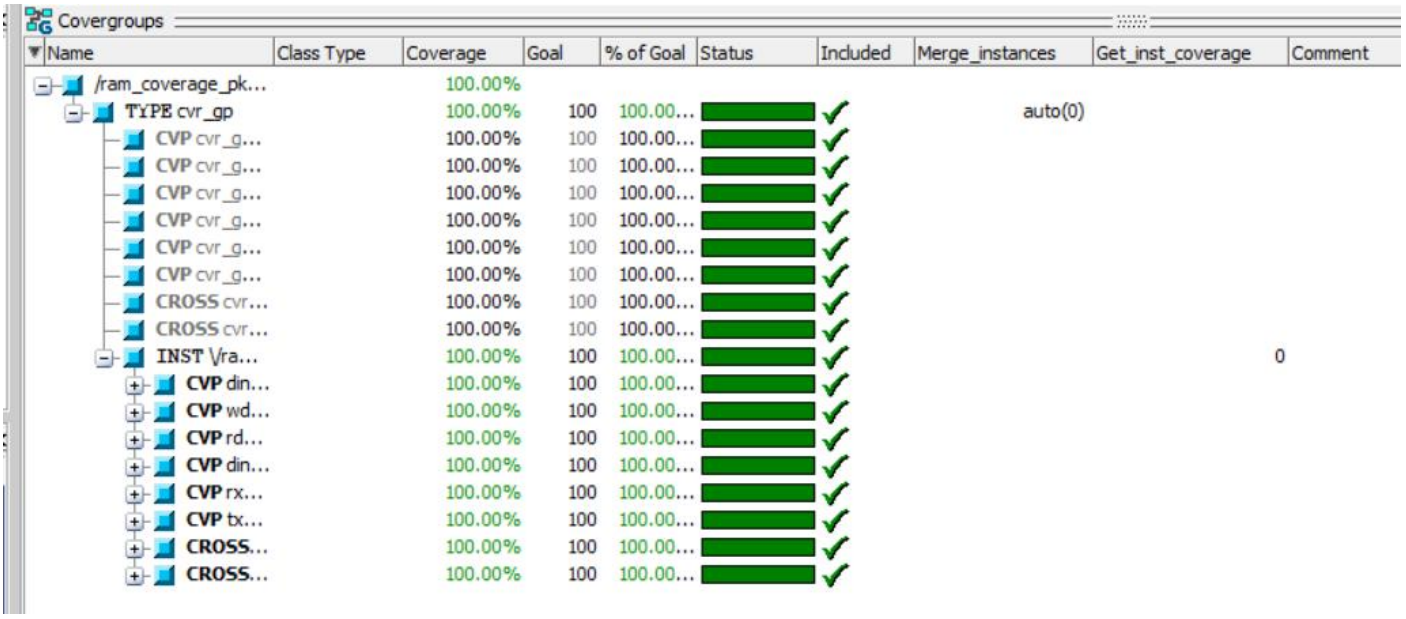
Label	Design Requirement Description	Stimulus Generation	Functional Coverage	Functionality Check
Ram_1	When the reset is asserted,the output dout=0 and tx_valid=0 should be low	Directed at the start of the simulation,then randomized with conistraint that drive the reset to be off 95% of the time		A checker in the scoreboard and bu using assertions to make sure the output is correct
Ram_2	When din[9:8] == 2'b00 the din[7:0] will be assigned to the write address pointer and if it equal to 2'b01 the din[7:0] will be written as a data input inside the RAM	Randamization under constraint on the rst_n signal to be off 95% of the time and when din[9:8] equal to 2'b00 the next operartion should be write address or write data and if it was equal to 2'b01 the next operation will be write address.	Cover the transition of din[9:8] from 0 to 1	A checker in the scoreboard and bu using assertions to make sure the output is correct
Ram_3	When din[9:8] == 2'b10 the din[7:0] will be assigned to the read address pointer and if it equal to 2'b11 the output dout will take the value from the RAM and tx_valid output will be high untill the end of the operation	Randamization under constraint on the rst_n signal to be off 95% of the time and when din[9:8] equal to 2'b01 the next operartion should be read data and if it was equal to 2'b11 the next operation will be read address.	Cover the transition of din[9:8] from 1 to 3	A checker in the scoreboard and bu using assertions to make sure the output is correct
Ram_4	When din[9:8] == 2'b00 the din[7:0] will be assigned to the write address pointer and if it equal to 2'b01 the din[7:0] will be written as a data input inside the RAM and, When din[9:8] == 2'b10 the din[7:0] will be assigned to the read address pointer and if it equal to 2'b11 the output dout will take the value from the RAM and tx_valid output will be high untill the end of the operation	Randamization under constraint on the rst_n signal to be off 95% of the time Every Write Address operation shall always be followed by either Write Address or Write Data operation. After a Write Data, the next operation shall be chosen with the following probability distribution: 60% → Read Address & 40% → Write Address Every Read Address operation shall always be followed by either Read Address or Read Data operation.After a Read Data, the next operation shall be chosen with the following probability distribution: 60% →	Cover the transition of din[9:8] to be as (write address => write data => read address => read data)	A checker in the scoreboard and bu using assertions to make sure the output is correct

6. Reports

- You can access all of the reports form this link : [Coverages_reports]

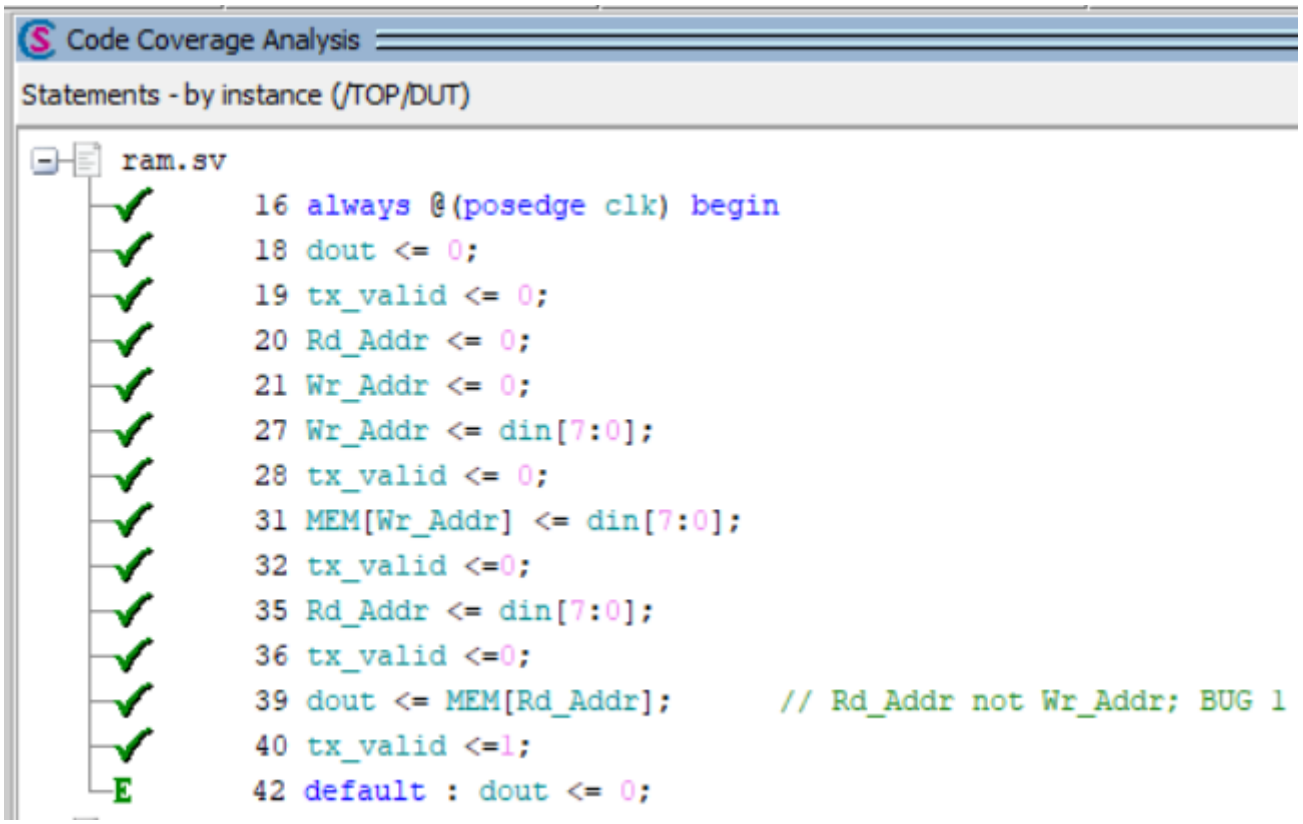
7. Questa snaps

7.1. Functional coverage

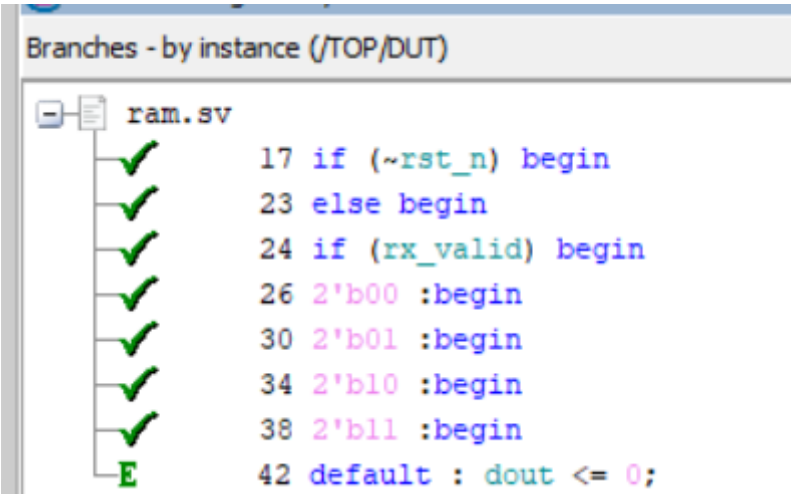


7.2. Code coverage

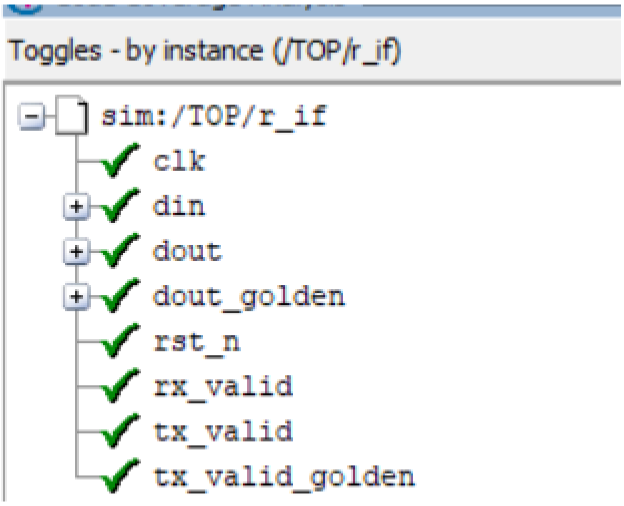
7.2.1. Statement coverage

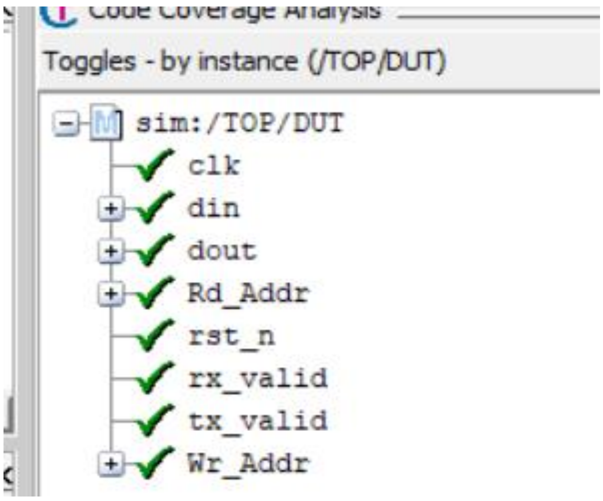


7.2.2. Branch coverage



7.2.3. Toggle coverage





7.2.4. Exclusion

- Default in the switch case was excluded because switch case covers all possible values, so it will never be executed.

7.3. **Assertions**

7.3.1. Assertions

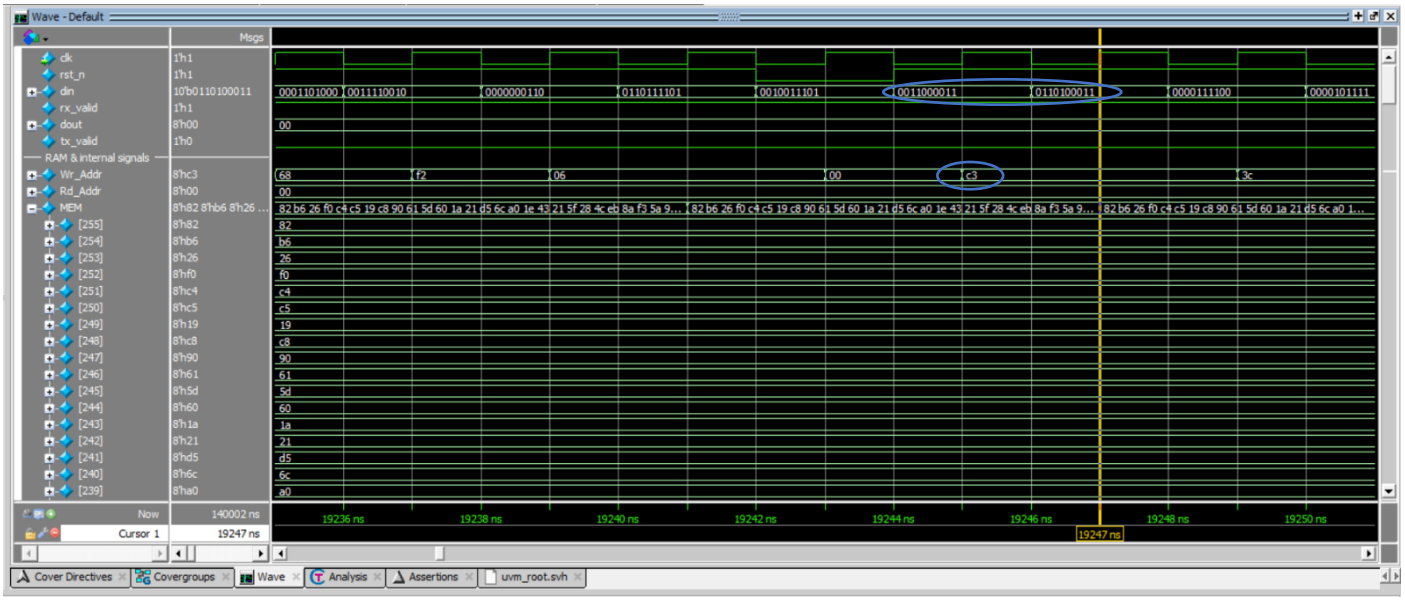
Assertions

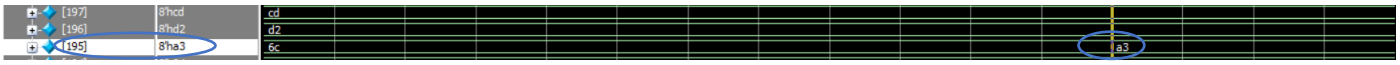
Name	Language	Enable	Failure Count	Assertion Type	Pass Count	Active Cou	Memory	Peak Memory	Peak Memory Time	Cumulative Threads	ATV	Assertion Expression	Included
/uvm_pkg::uvm_reg_ma...	SVA	on	0	Immediate	0	-	-	-	-		off	assert ($cast(seq,o))	✗
/uvm_pkg::uvm_reg_ma...	SVA	on	0	Immediate	0	-	-	-	-		off	assert ($cast(seq,o))	✗
/ram_sequence_pkg::ra...	SVA	on	0	Immediate	1	-	-	-	-		off	assert (randomize(...))	✓
/ram_sequence_pkg::ra...	SVA	on	0	Immediate	1	-	-	-	-		off	assert (randomize(...))	✓
/ram_sequence_pkg::ra...	SVA	on	0	Immediate	1	-	-	-	-		off	assert (randomize(...))	✓
/TOP/DUT/ram_assertion...	SVA	on	0	Concurrent	1	-	0B	0B	0 ns	0	off	assert(@(posedge clk) (~rst_...	✓
/TOP/DUT/ram_assertion...	SVA	on	0	Concurrent	1	-	0B	0B	0 ns	0	off	assert(disable iff (~rst_n) (@...	✓
/TOP/DUT/ram_assertion...	SVA	on	0	Concurrent	1	-	0B	0B	0 ns	0	off	assert(disable iff (~rst_n) (@...	✓
/TOP/DUT/ram_assertion...	SVA	on	0	Concurrent	1	-	0B	0B	0 ns	0	off	assert(disable iff (~rst_n) (@...	✓
/TOP/DUT/ram_assertion...	SVA	on	0	Concurrent	1	-	0B	0B	0 ns	0	off	assert(disable iff (~rst_n) (@...	✓

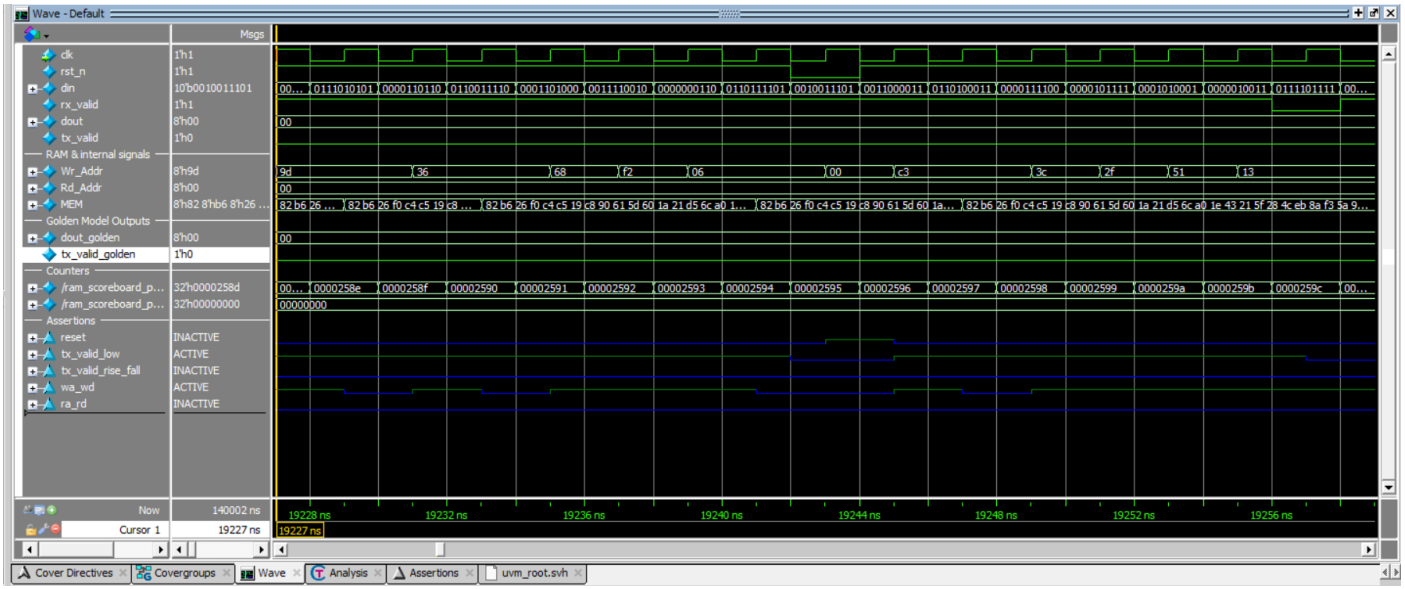
7.3.2. Cover directives of the assertions

Name	Language	Enabled	Log	Count	AtLeast	Weight	Cmplt %	Cmplt graph	Limit	Included	Memory	Peak Memory	Peak Memory Time	Cumulative Threads
/TOP/DUT/ram_ass...	SVA	✓	Off	17542	1	1	100%		Unli...	✓	0	0	0 ns	0
/TOP/DUT/ram_ass...	SVA	✓	Off	18937	1	1	100%		Unli...	✓	0	0	0 ns	0
/TOP/DUT/ram_ass...	SVA	✓	Off	10805	1	1	100%		Unli...	✓	0	0	0 ns	0
/TOP/DUT/ram_ass...	SVA	✓	Off	48623	1	1	100%		Unli...	✓	0	0	0 ns	0
/TOP/DUT/ram_ass...	SVA	✓	Off	3532	1	1	100%		Unli...	✓	0	0	0 ns	0

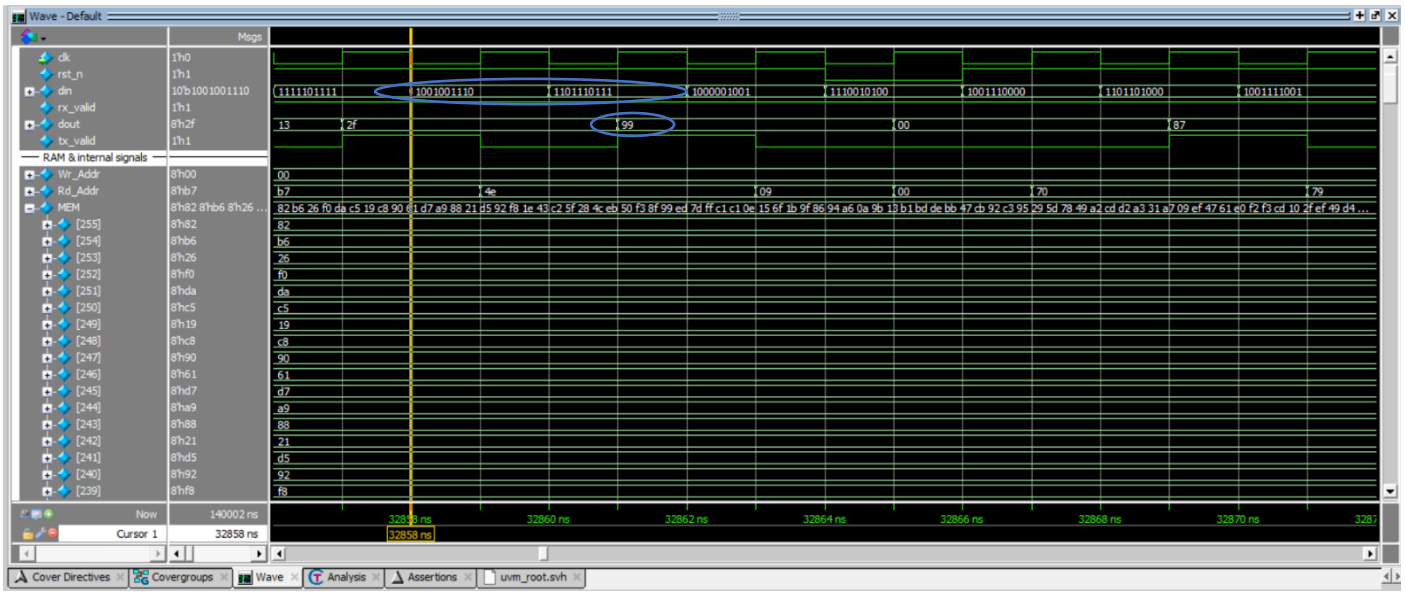
7.4. Write Simulation

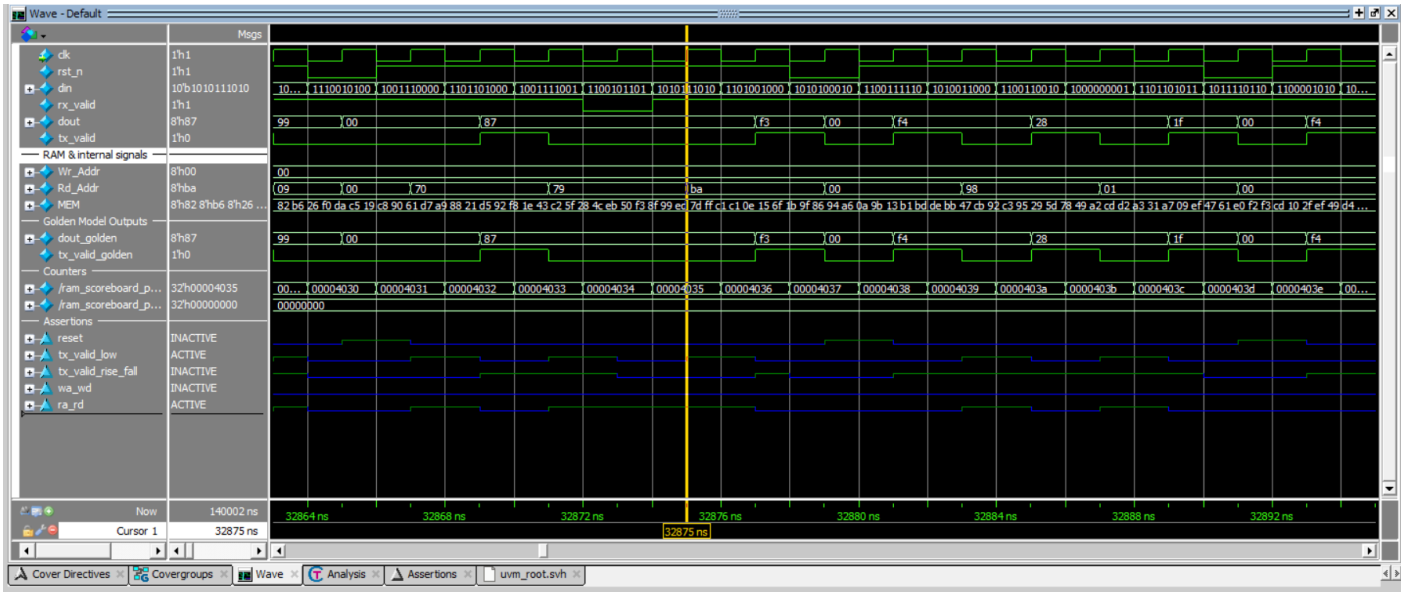




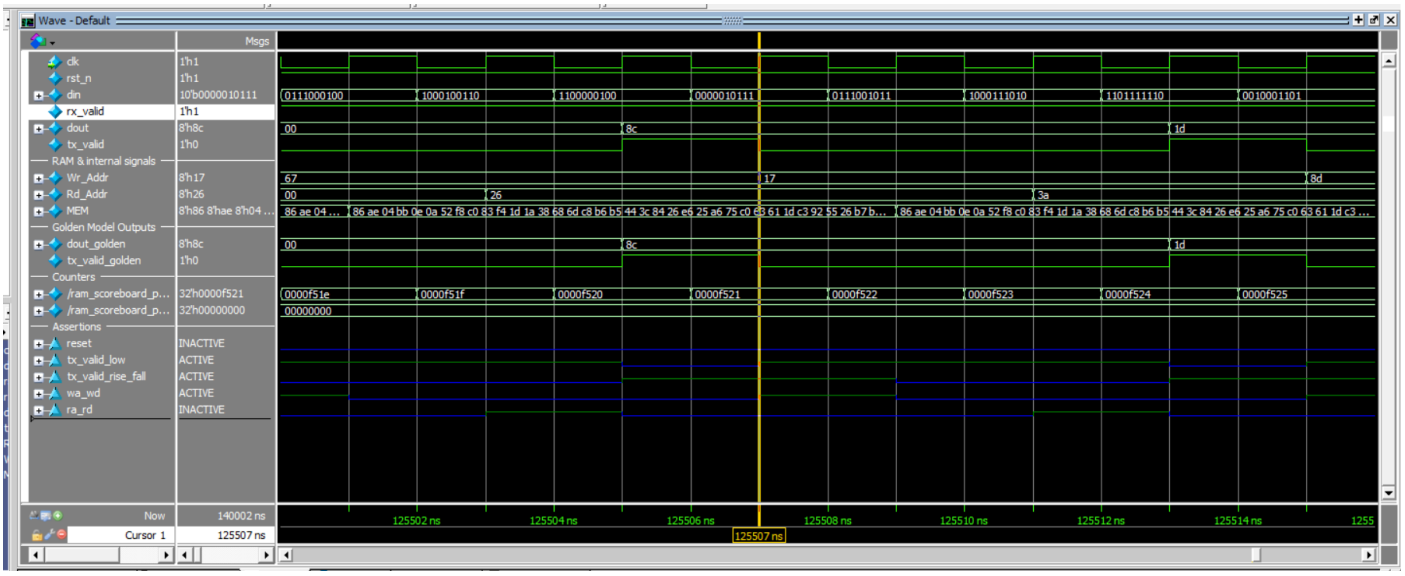


7.5. Read Simulation

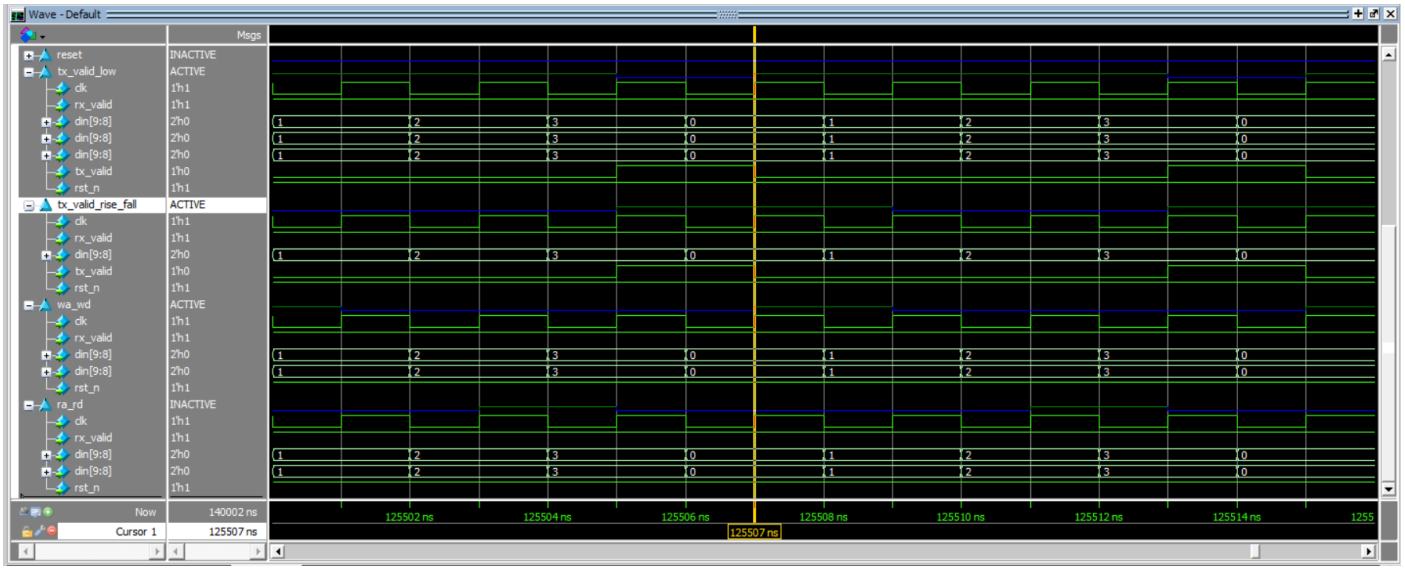




7.6. Read and Write simulation



7.7. Assertions



7.8. Transcript

Transcript

```
# UVM_INFO ram_test.sv(44) @ 0: uvm_test_top [run_phase] reset generation start
# ****************************************************************************
# * Questa UVM Transaction Recording Turned ON.                              *
# * recording_detail has been set.                                           *
# *  To turn off, set 'recording_detail' to off:                             *
# * uvm_config_db#(int)            ::set(null, "", "recording_detail", 0); *
# * uvm_config_db#(uvm_bitstream_t)::set(null, "", "recording_detail", 0); *
# ****************************************************************************
# UVM_INFO ram_test.sv(46) @ 2: uvm_test_top [run_phase] reset deasserted
# UVM_INFO ram_test.sv(48) @ 2: uvm_test_top [run_phase] WRITE generation start
# UVM_INFO ram_test.sv(50) @ 20002: uvm_test_top [run_phase] WRITE generation END
# UVM_INFO ram_test.sv(52) @ 20002: uvm_test_top [run_phase] READ generation start
# UVM_INFO ram_test.sv(54) @ 40002: uvm_test_top [run_phase] READ generation end
# UVM_INFO ram_test.sv(56) @ 40002: uvm_test_top [run_phase] READ AND WRITE generation start
# UVM_INFO ram_test.sv(58) @ 140002: uvm_test_top [run_phase] READ AND WRITE generation end
# UVM_INFO verilog_src/uvm-1.1d/src/base/uvm_objection.svh(1267) @ 140002: reporter [TEST_DONE] 'run' phase is ready to proceed to the 'extract' phase
# UVM_INFO ram_scoreboard.sv(50) @ 140002: uvm_test_top.env.sb [report_phase] correct_count =       70001
# UVM_INFO ram_scoreboard.sv(51) @ 140002: uvm_test_top.env.sb [report_phase] error_count =           0
#
# --- UVM Report Summary ---
#
# ** Report counts by severity
# UVM_INFO :   14
# UVM_WARNING :    2
# UVM_ERROR :    0
# UVM_FATAL :    0
# ** Report counts by id
# [Questa UVM]     2
# [RNTST]     1
# [TEST_DONE]     1
# [UVM_DEPRECATED]     2
# [report_phase]     2
# [run_phase]     8
# ** Note: $finish    : C:/questasim64_2021.1/win64/../verilog_src/uvm-1.1d/src/base/uvm_root.svh(430)
#    Time: 140002 ns  Iteration: 61  Instance: /TOP
```